

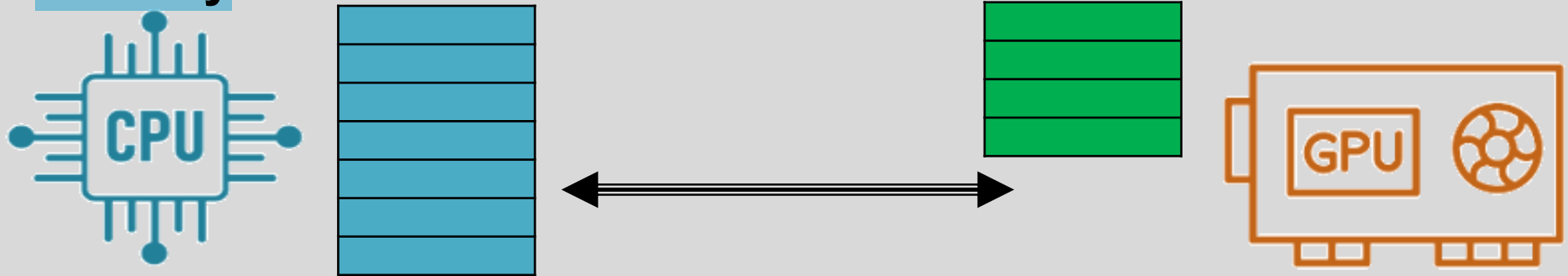
GPU Programming

Ernesto Sanchez

GPU Programming model

GPGPU Programming model

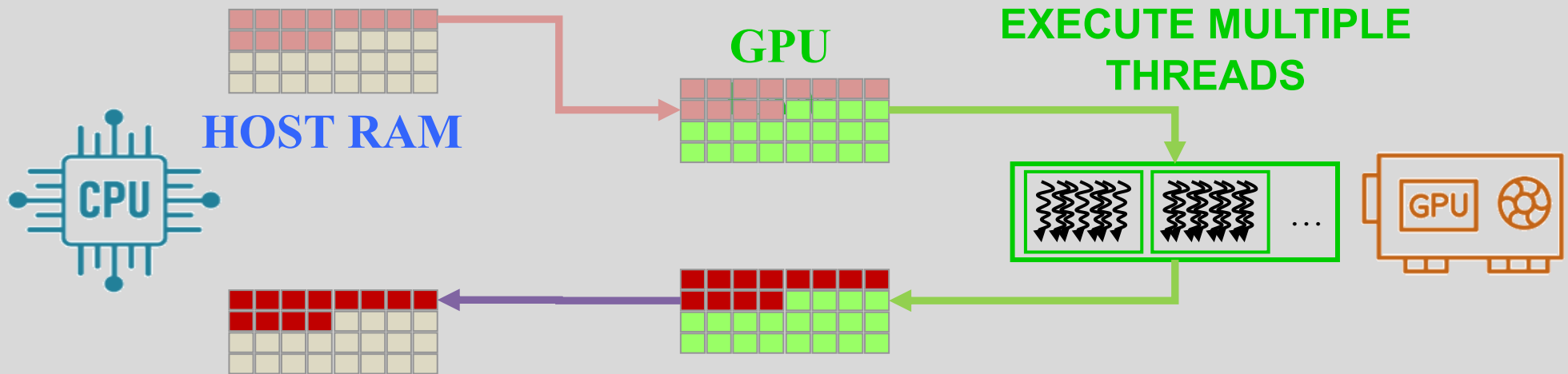
- Serial parts of a program, or those with low level of parallelism, keep running on the **CPU** (HOST)
- Computational-intensive data-parallel regions are executed on the **GPU** (device)
- Required data is moved on GPU memory and back to HOST memory



GPU Programming model

Data movement

- Data must be moved from HOST to DEVICE memory to be processed on the GPU
- When data is processed, and no more needed on the GPU, it is transferred back to HOST



Basic CUDA programming model

How to use the GPU to speed up a program?

- Is there any level of parallelism?
- **Configure device** (libraries, drivers, toolkit)
- **Initialize the GPU's memory** (application type, data)
- **Define the total number of concurrent threads to be launch in the GPU**
(Grid, Thread blocks, Warps, Threads)
- **Execute the program in the GPU** (one or more Kernels)
- **Collect/retrieve the results to the Host**

Linking physical architecture to programming model

- **Thread**

- Distributed by the CUDA runtime

(threadId.x)

- **Block**

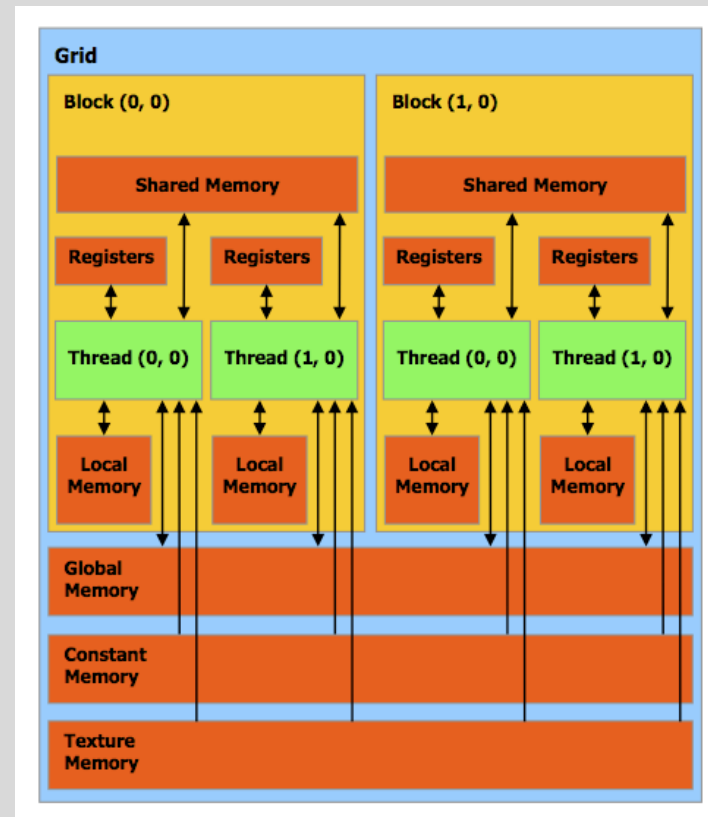
- A user defined group of 1 to ~1024 threads

(blockId.x)

- **Grid**

- A group of one or more blocks.
 - A grid is created for each CUDA kernel function called.

Imagine the thread organization as an array of thread indices. It is possible to assume multidimensional arrays for blocks and threads.



Linking physical architecture to programming model

CUDA provides built in multidimensional **thread indexing** capabilities with a struct called **dim3**.

dim3 is a struct defined in vector_types.h .

dim3 helps to define a 1D, 2D or 3D structures:

1D $\rightarrow$ (N)

2D $\rightarrow$ (Nx, Ny)

3D $\rightarrow$ (Nx, Ny, Nz)

Define the grid and block according to the problem necessities.

- dim3 grid(Nx, Ny, Nz);
- dim3 block(Nx, Ny, Nz);

Grid (X, Y, Z)

└─ Block (X, Y, Z)

└─ Thread (X, Y, Z)

Linking physical architecture to programming model

Example:

- `dim3 grid(256);` // defines a grid of 256 x 1 x 1 blocks
- `dim3 block(32, 32);` // defines a block of 32 x 32 x 1 threads

Use the defined structures when calling the kernel:

- `my_kernel<<<grid, block>>>(...);`

`blockId.x`
`(blockDim.x, blockDim.y)`
`(threadId.x, threadId.y)`

```
my_kernel<<<(256,1) , (32,32,1)>>>(...);
```

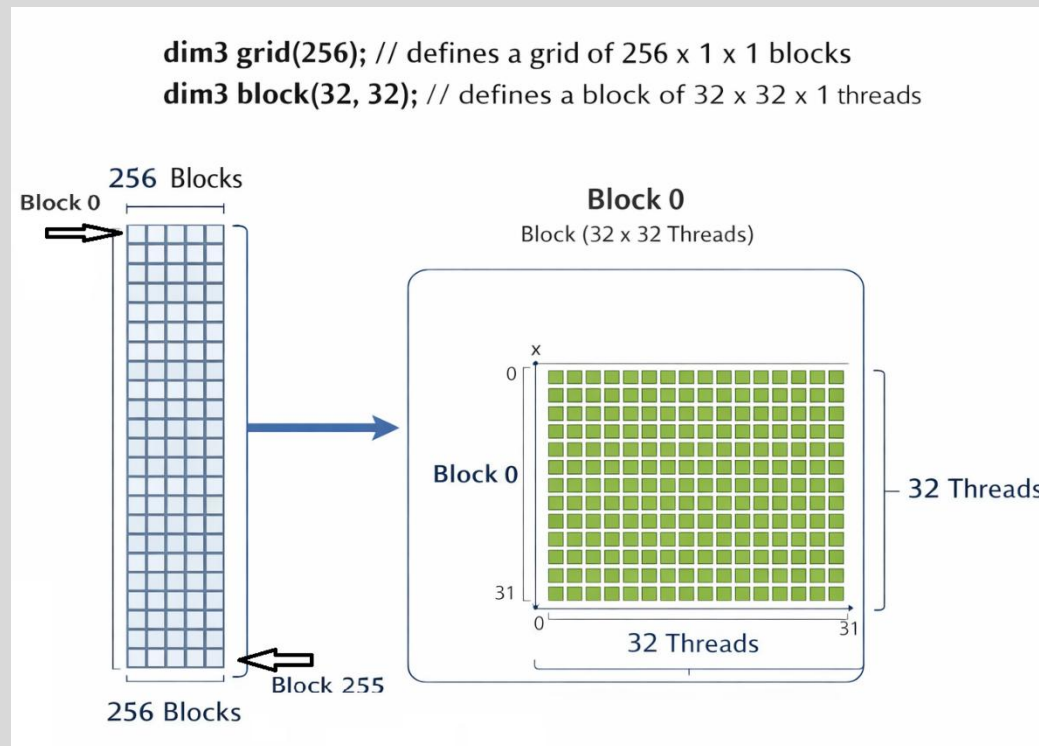
```
blockDim.x = 32    0 <= threadIdx.x < 32
```

```
0 <= blockDim.x < 256    blockDim.y = 32    0 <= threadIdx.y < 32
```

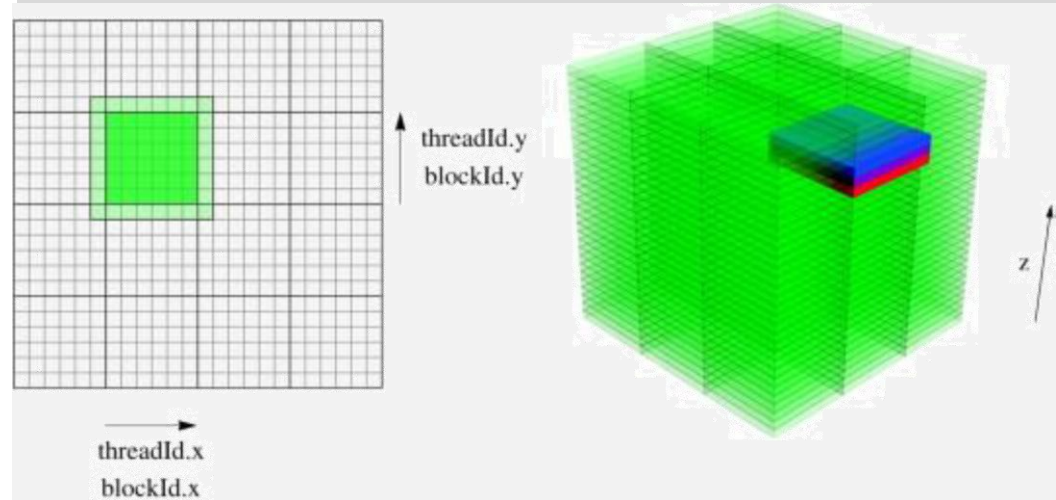
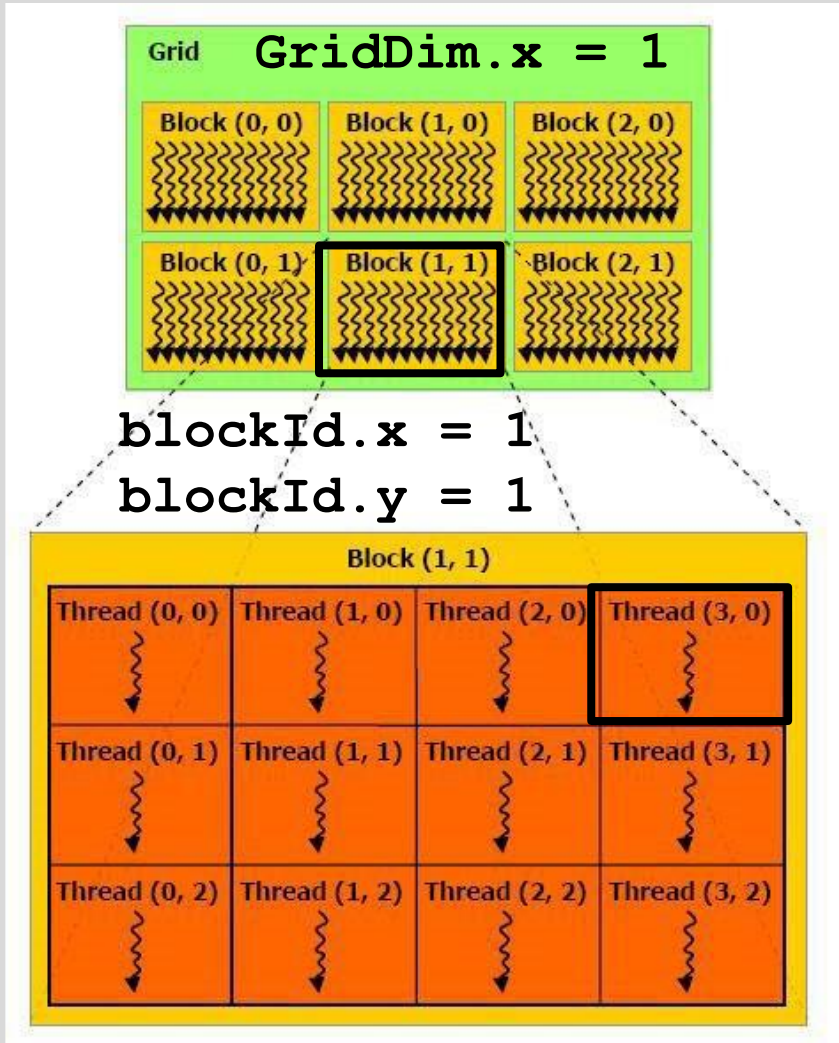
Linking physical architecture to programming model

Example:

- `dim3 grid(256);` // defines a grid of 256 x 1 x 1 blocks
- `dim3 block(32, 32);` // defines a block of 32 x 32 x 1 threads



Linking physical architecture to programming model



```
dim3 grid(3,2);  
dim3 block(4,3);
```

```
blockDim.x = 4  
blockDim.y = 3
```

```
ThreadId.x = 3  
ThreadId.y = 0
```

Linking physical architecture to programming model

Unique thread Indices

- In general, the main target of the programmer is to define the execution process of one of the threads in the program.
- Then, the programmer uses the indices to generalize the parallel execution and access data in memory according to the application.

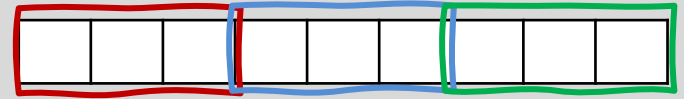
1D example

One-dimension arrays:

```
idx = blockDim.x * blockIdx.x + threadIdx.x;
```

Example:

```
my_kernel<<<(3,1) , (3,1,1)>>> (...);
```



```
blockDim.x = 3  
blockIdx.x = [0, 1, 2]  
threadIdx.x = [0, 1, 2]
```

Idx ₁ =	(3*0)	+	0	=	0	↙
Idx ₂ =	(3*0)	+	1	=	1	↙
Idx ₃ =	(3*0)	+	2	=	2	↙
Idx ₄ =	(3*1)	+	0	=	3	↙
Idx ₅ =	(3*1)	+	1	=	4	↙
Idx ₆ =	(3*1)	+	2	=	5	↙
Idx ₇ =	(3*2)	+	0	=	6	↙
Idx ₈ =	(3*2)	+	1	=	7	↙
Idx ₉ =	(3*2)	+	2	=	8	↙

2D example

Two-dimension arrays:

```
dim3 grid(32,2,1);  
dim3 block(1024,1,1);  
my_kernel<<< grid,block >>>(...);
```

Example 2:

```
Block_Idx = GridDim.x * blockIdx.y + blockIdx.x;  
idx = Block_Idx*(blockDim.x) + threadIdx.x;
```

```
gridDim.x = 32  
gridDim.y = 2  
blockIdx.x = [0,31]  
blockIdx.y = [0,1]  
blockDim.x = 1024  
blockDim.y = 1  
threadIdx.x = [0, 1023]
```

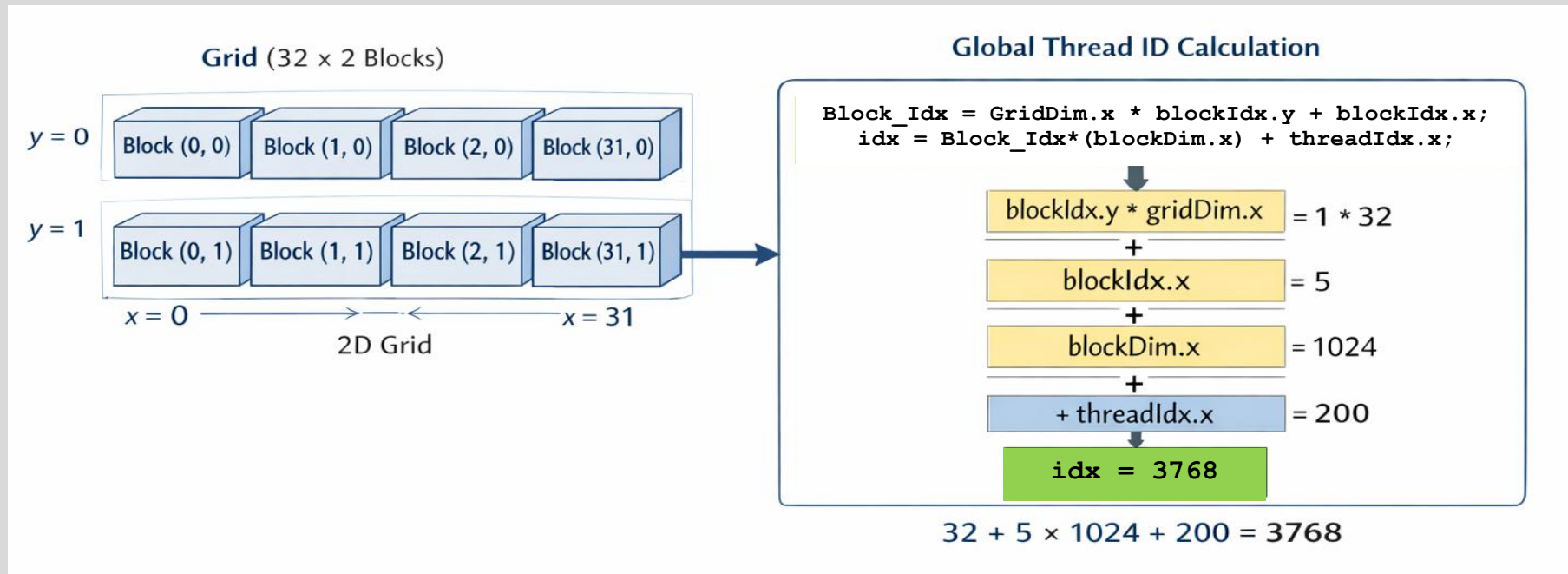
Total Threads:
 $32 \times 2 \times 1024 = 65,536$

2D example

Two-dimension arrays:

```
dim3 grid(32,2,1);  
dim3 block(1024,1,1);  
my_kernel<<< grid,block >>> (...);
```

Example 2:



SCALABILITY

Transparent scalability:

The ability of an algorithm to operate on data of different sizes without reprogramming and user tuning

- Maximize the number of threads in case the data is large
 - both the block and grid dimensions have maximum values
 - launch each block with as many threads as possible

- Example:

Launch **101,500** threads with a maximum block dimension of **1,024**

requires between 99 and 100 blocks: $\frac{101,500}{1,024} = 99,12$

if we use **100** blocks, there are too many: $100 \times 1024 = 102,400$

launch more threads, then check for excess threads in the kernel

SCALABILITY

Non so se da qui in poi sono a guardare

Transparent scalability:

```
__global__ void kernelFunc(float* A, float* B, size_t N)
{
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i >= N)
        return;
    ... //do stuff
}
```

SCALABILITY

Transparent scalability:

- Example: Solve $C = A + B$ where A , B , and $C \in \mathbb{R}^N$

N is unknown at compile time

- but can be big (which is kind of the point)

Use the maximum number of **threads per block**

- Allows the largest possible matrix
- Limits to block dimension ($\approx 1,024$ due to SM) and grid dimension ($\approx 65,535$)
- How many blocks should be launched?

`blocks = ceil((double)N/threads);` `// round up`

- or

`blocks = N/threads + 1;` `// sometimes launches extra block`

$$\begin{bmatrix} C_1 \\ \vdots \\ C_N \end{bmatrix} = \begin{bmatrix} A_1 \\ \vdots \\ A_N \end{bmatrix} + \begin{bmatrix} B_1 \\ \vdots \\ B_N \end{bmatrix}$$

SCALABILITY

Transparent scalability:

- Example: Solve $C = A + B$ where A , B , and $C \in \mathbb{R}^N$

```
__global__ void kernelAdd(float* C, float* A, float* B, size_t N){  
    //calculate the index into the array  
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;  
    if(i >= N) return; //exit if the thread is hanging  
    C[i] = A[i] + B[i]; //perform the addition  
}
```

Not all array sizes will fit perfectly onto a grid

- Some launched threads will not have corresponding array values
- Test for “hanging” threads so that you don’t cause a segmentation fault

segmentation fault: memory access violation, or general protection fault

SCALABILITY

Transparent scalability:

- Example: Solve $C = A + B$ where A , B , and $C \in \mathbb{R}^N$

```
__global__ void kernelAdd(float* C, float* A, float* B, size_t N){  
    //calculate the index into the array  
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;  
    if(i >= N) return; //exit if the thread is hanging  
    C[i] = A[i] + B[i]; //perform the addition  
}
```

Not all array sizes will fit perfectly onto a grid

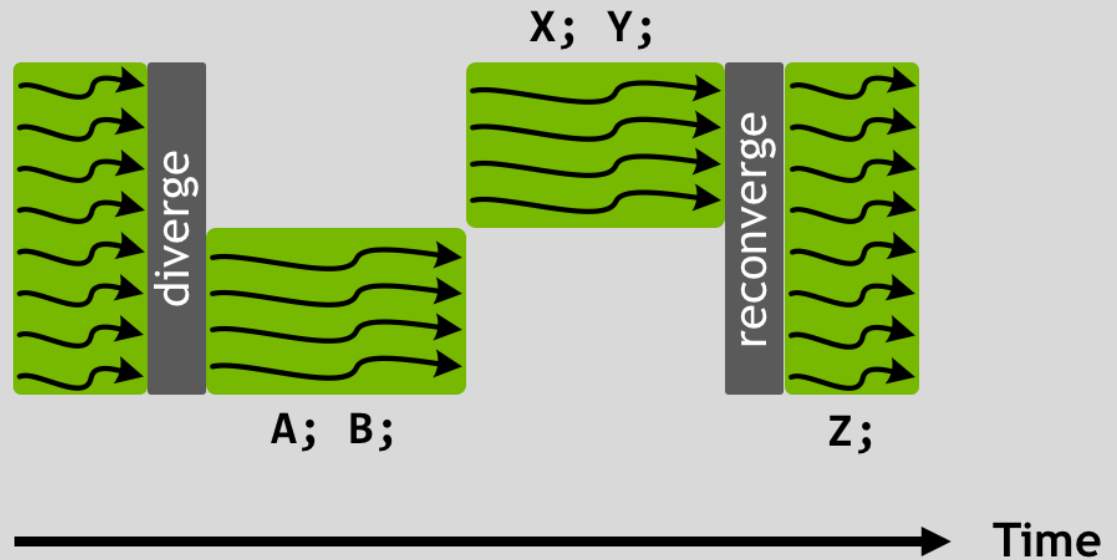
- Some launched threads will not have corresponding array values
- Test for “hanging” threads so that you don’t cause a segmentation fault

segmentation fault: memory access violation, or general protection fault

SCALABILITY

Transparent scalability: Divergence

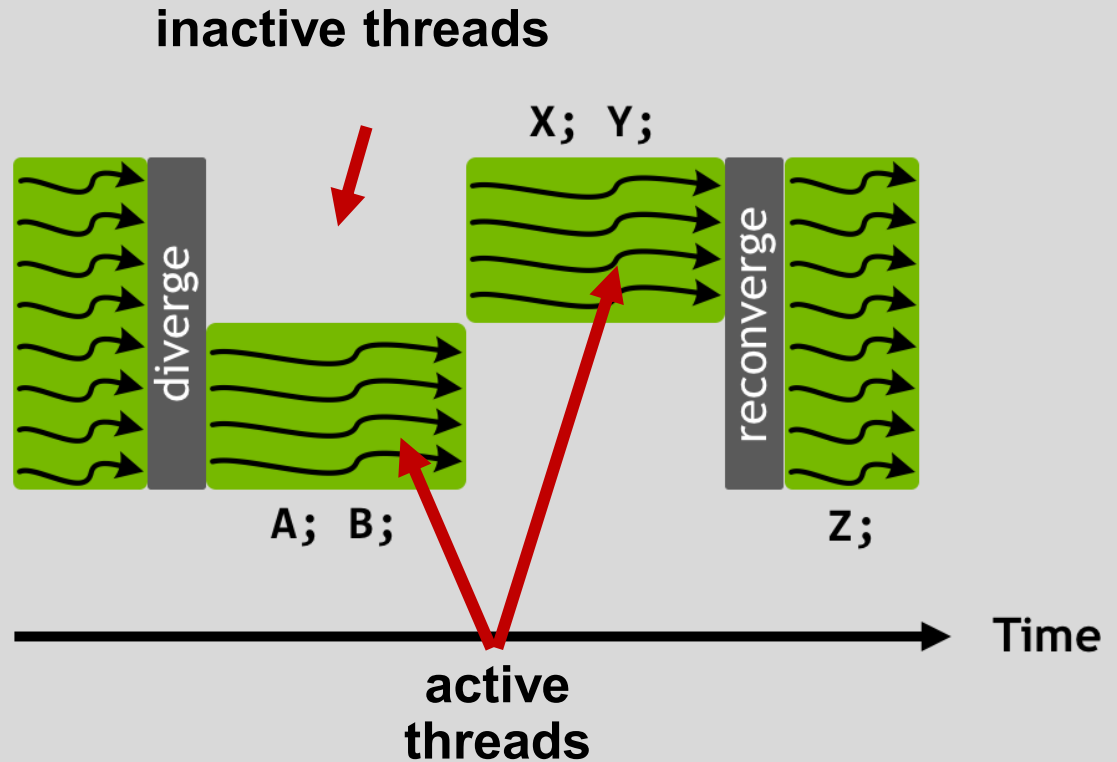
```
__global__ void foo(...)  
{  
...  
    if (ThreadIdx.x < 4)  
    {  
        A;  
        B;  
    }  
    else  
    {  
        X;  
        Y;  
    }  
...  
}
```



SCALABILITY

Transparent scalability: Divergence

```
__global__ void foo(...)  
{  
...  
    if (ThreadId.x < 4)  
    {  
        A;  
        B;  
    }  
    else  
    {  
        X;  
        Y;  
    }  
...  
}
```



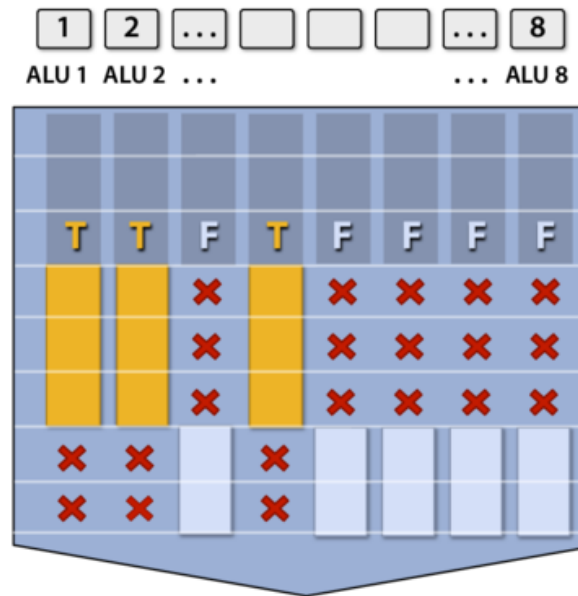
SCALABILITY

Transparent scalability: Divergence

```
<unconditional code>
float x = A[i];
if (x > 0) {
    float tmp = exp(x, 5.f);
    tmp *= kMyConst1;
    x = tmp + kMyConst2;
} else {
    float tmp = kMyConst1;
    x = 2.f * tmp;
}
<resume unconditional code>

result[i] = x;
```

Time (clocks)



Not all ALUs do useful work!

Worst case: 1/8 peak performance

SCALABILITY

Transparent scalability: Thread synchronization

When several flow paths are executed, thread synchronization (barrier) is advised

```
//synchronize the threads at block level  
__syncthreads();
```

In cooperative groups:

```
#include <cuda_runtime_api.h>  
#include <cuda.h>  
#include <cooperative_groups.h>  
  
cooperative_groups::grid_group g = cooperative_groups::this_grid();  
g.sync();
```

SCALABILITY

Transparent scalability:

Device functions:

Large applications cannot resort on individual kernels, so several kernels are described and parallel functions (device functions)

- can only be called from other **device** or **global** functions
- **__device__** functions cannot be called from host code.

```
__global__ void calledFromCpuForGPU(...)  
{  
    //This function is called by CPU for execution on GPU  
}
```

```
__device__ void calledFromGPUforGPU(...)  
{  
    // This function is called by GPU for execution on GPU  
}
```